BTS SIO · option SISR · 1re année

RP6

Serveur web, base de données et HTTPS

Debian 13 · Apache · MariaDB · PHP · pfSense

OBJECTIF

Mettre en place un site intranet sur un serveur Debian : Apache affiche les données d'une base MariaDB grâce à PHP, puis le site est chiffré en HTTPS avec un certificat signé par l'autorité de certification interne créée dans pfSense.

RÉALISÉ PAR	**AIT TAHAR Menzo**
FORMATION	BTS SIO, option SISR — 1re année
RÉALISATION	RP6 · Serveur web, BDD et HTTPS
DATE	19 octobre 2026
MAQUETTE	pfSense 2.6 · serveur Debian 13 (192.168.1.10) · client Windows 11 (192.168.1.4)

Sommaire

Découverte - observer....3
Réflexion - à répondre par écrit....3
Apport technique - repères....4
Application A - Apache....4
Application B - la base de données....7
Application C - HTTPS....11
Vérification....18
Rendu attendu....18
Conclusion....21

Les numéros de page se mettent à jour dans Word : clic droit sur le sommaire, puis « Mettre à jour les champs ».

RP6 BTS SIO SISR | 1re année

Serveur web, BDD et HTTPS

Debian + Apache + MariaDB | un site dynamique et chiffré

CONTEXTE

L'entreprise veut un site interne qui affiche ses données, sans que rien ne circule en clair. Vous allez monter un serveur web Apache relié à une base de données, puis le chiffrer.

PRÉREQUIS	DURÉE	MODALITÉ	À PRODUIRE
Maquette du RP5	**4 heures**	**Binôme, 1 rapport / étudiant**	**1 site HTTPS + 1 base + rapport**

1 Découverte - observer — 20 min

BUT DE LA PHASE

Comprendre comment un navigateur décide de faire confiance à un site. Le cadenas indique que la connexion est chiffrée (HTTPS) et permet de voir qui a émis le certificat : une autorité de certification (CA) que le navigateur connaît déjà.

Depuis le client, ouvrez un site en HTTPS et cliquez sur le cadenas.

Site	Protocole	Qui a émis le certificat ?
Un site public en HTTPS	HTTPS / TLS	Une CA publique reconnue par le navigateur (Let's Encrypt, DigiCert, Sectigo, Google Trust Services… : relève la vraie)
Votre Debian (après la phase 4)	HTTPS / TLS	D'abord le serveur lui-même (certificat auto-signé « debian »), puis CA-Entreprise, la CA interne créée dans pfSense

2 Réflexion - à répondre par écrit — 15 min

BUT DE LA PHASE

Réfléchir aux choix de sécurité avant de configurer : séparer les rôles, limiter les droits de chaque compte et comprendre ce qui rend un certificat digne de confiance.

Pourquoi séparer le site (Apache) et les données (base) ?

Pour des raisons de sécurité, de modularité et de performance. Si le serveur web est compromis, les données stockées sur un serveur ou un service de base de données cloisonné restent mieux protégées. Cela permet aussi d'évoluer (faire porter la base de données sur une autre machine si la charge augmente).

Pourquoi un utilisateur dédié pour la base plutôt que root ?

Par application du principe du moindre privilège. Si une vulnérabilité SQL s'exprime sur le site web, l'attaquant n'aura accès qu'à l'utilisateur restreint de la base de données et non aux pleins pouvoirs du compte root (qui permettrait de modifier ou supprimer l'ensemble des bases du système).

Pourquoi le navigateur se méfie-t-il d'un certificat auto-signé ?

Parce qu'il ne provient pas d'une Autorité de Certification (CA) reconnue et pré-enregistrée dans le magasin de confiance du système ou du navigateur. Il ne peut donc pas garantir de manière certifiée l'identité réelle du serveur (risque d'attaque de type Man-in-the-Middle).

3 Apport technique - repères — 15 min

BUT DE LA PHASE

Connaître les briques utilisées et leurs ports avant de les installer. Cela aide ensuite à vérifier chaque service et à savoir quel flux autoriser.

Point	À retenir
Apache	Serveur web. Port 80 (HTTP) et 443 (HTTPS)
MariaDB	Stocke les données. Port 3306, en local par défaut
Certificat	Prouve l'identité du serveur, signé par une autorité (CA)
CA interne	L'autorité de l'entreprise, à installer sur les postes

4 Application A - Apache — 40 min

BUT DE LA PHASE

Installer le serveur web qui va héberger le site intranet de l'entreprise, puis vérifier qu'il répond, d'abord sur le serveur lui-même, ensuite depuis le poste client.

☒ Installer Apache sur Debian

Pourquoi ? *Apache est le logiciel serveur web : il reçoit les demandes des navigateurs et leur renvoie les pages du site, sur le port 80 en HTTP.*

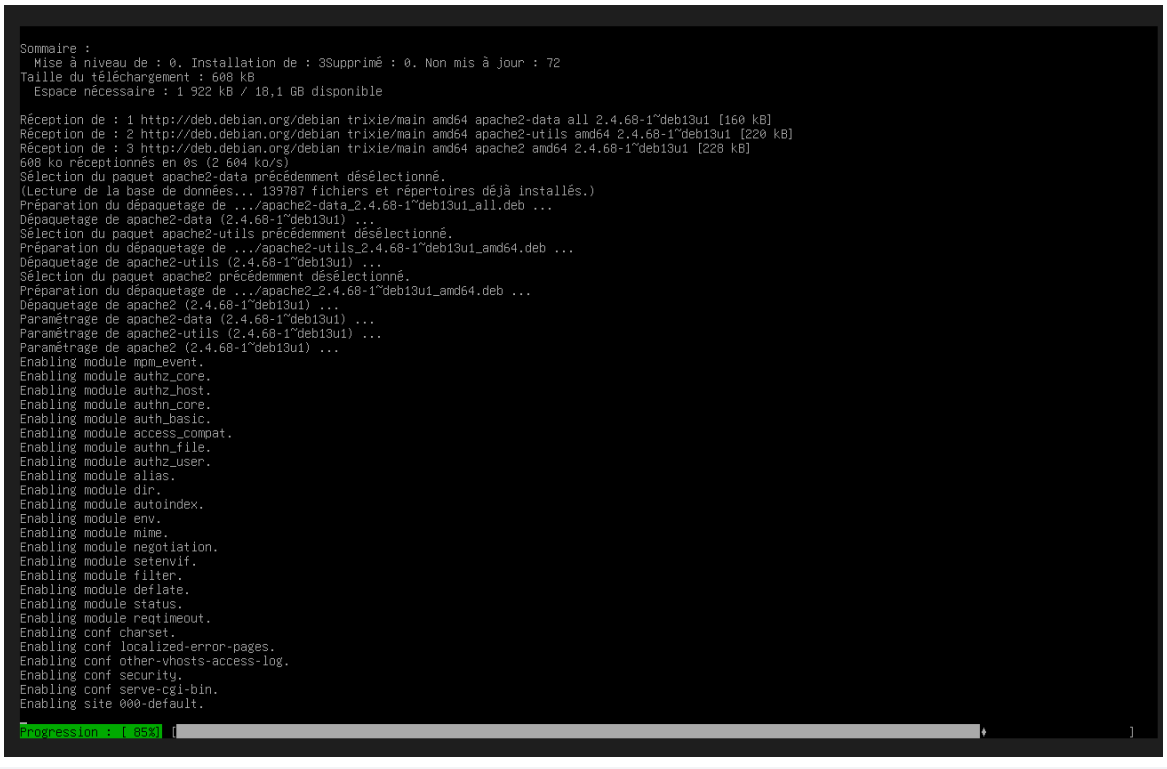


Figure 1 – Installation d'Apache
La commande « apt install apache2 » installe le serveur web Apache sur le Debian. À la fin, Apache active ses modules de base et le site par défaut (000-default).

```
root@debian: # apt install curl -y
Mis à jour :
  libcurl3t64-gnutls  libcurl4t64

Installation de :
  curl

Sommaire :
  Mise à niveau de : 2. Installation de : 1Supprimé : 0. Non mis à jour : 73
Taille du téléchargement : 1 046 kB
  Espace nécessaire : 507 kB / 18,0 GB disponible

Réception de : 1 http://security.debian.org/debian-security trixie-security/main amd64 libcurl4t64 amd64 8.14.1-2+deb13u7 [392
Réception de : 2 http://security.debian.org/debian-security trixie-security/main amd64 curl amd64 8.14.1-2+deb13u7 [270 kB]
Réception de : 3 http://security.debian.org/debian-security trixie-security/main amd64 libcurl3t64-gnutls amd64 8.14.1-2+deb13u
1 046 ko réceptionnés en 0s (3 676 ko/s)
apt-listchanges : Lecture des fichiers de modifications (« changelog »)...
(Lecture de la base de données... 140327 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../libcurl4t64_8.14.1-2+deb13u7_amd64.deb ...
Dépaquetage de libcurl4t64:amd64 (8.14.1-2+deb13u7) sur (8.14.1-2+deb13u5) ...
Sélection du paquet curl précédemment désélectionné.
Préparation du dépaquetage de .../curl_8.14.1-2+deb13u7_amd64.deb ...
Dépaquetage de curl (8.14.1-2+deb13u7) ...
Préparation du dépaquetage de .../libcurl3t64-gnutls_8.14.1-2+deb13u7_amd64.deb ...
Dépaquetage de libcurl3t64-gnutls:amd64 (8.14.1-2+deb13u7) sur (8.14.1-2+deb13u5) ...
Paramétrage de libcurl4t64:amd64 (8.14.1-2+deb13u7) ...
Paramétrage de libcurl3t64-gnutls:amd64 (8.14.1-2+deb13u7) ...
Paramétrage de curl (8.14.1-2+deb13u7) ...
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
Traitement des actions différées (« triggers ») pour libc-bin (2.41-12+deb13u4) ...
root@debian:~# curl -I http://localhost
HTTP/1.1 200 OK
Date: Mon, 19 Oct 2026 07:56:47 GMT
Server: Apache/2.4.69 (Debian)
Content-Type: text/html;charset=UTF-8

root@debian:~# _
```

Figure 2 – Test d'Apache sur le serveur
On installe l'outil curl, puis « curl -I http://localhost » interroge Apache. La réponse « HTTP/1.1 200 OK » prouve que le serveur web répond bien sur le port 80.

☒ Mettre vos noms sur la page d'accueil

Pourquoi ? *Remplacer la page par défaut permet de vérifier que c'est bien notre serveur qui répond, et de personnaliser le site.*

```
  GNU nano 8.4                                        /var/www/html/index.html
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="utf-8"><title>Intranet RP6</title></head>
<body>
<H1>Intranet de l'entreprise</h1>>
<p>Realise par AIT TAHAR Menzo</p>
</body>
</html>
```

Figure 3 – Page d'accueil personnalisée
Le fichier /var/www/html/index.html est modifié avec nano : il affiche le titre « Intranet de l'entreprise » et le nom de l'auteur.

```
root@debian:~# apt install openssh-server -y
openssh-server est déjà la version la plus récente (1:10.0p1-7+deb13u4).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 73
root@debian:~#
```

Figure 4 – Serveur SSH
« apt install openssh-server » indique que SSH est déjà installé. SSH permet de se connecter au Debian depuis le client Windows pour copier-coller les commandes et transférer des fichiers.

☒ Le client n'affiche pas le site ? Ouvrir le flux dans pfSense

Pourquoi ? *pfSense filtre le trafic qui passe d'un réseau à un autre. Ici, le client et le serveur sont sur le même réseau LAN : le trafic ne traverse pas le pare-feu, donc aucune règle n'a été nécessaire. Si le serveur était dans la zone SERVEURS (192.168.2.0/24), il faudrait autoriser les ports 80 et 443 depuis le LAN.*

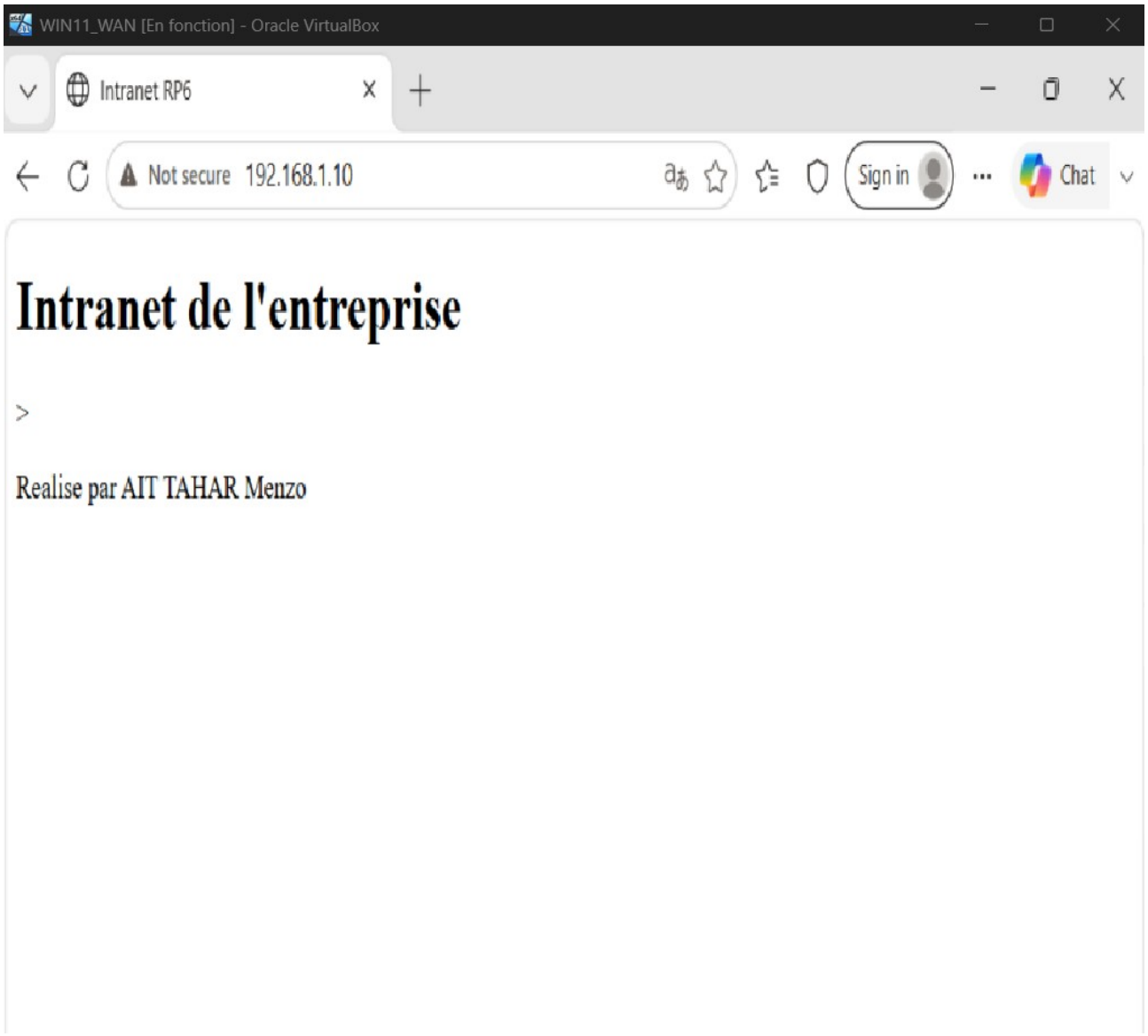


Figure 5 – La page vue depuis le client
Le navigateur du client Windows 11 affiche la page à l'adresse http://192.168.1.10. Le client et le serveur sont sur le même réseau LAN de pfSense : aucune règle de pare-feu n'a été nécessaire. Le « > » sous le titre vient d'une faute de frappe dans le code (« </h1>> »).

5 Application B - la base de données 1 h

BUT DE LA PHASE

Stocker les données de l'entreprise dans une base MariaDB et les afficher sur le site grâce à PHP. La sécurité repose sur deux règles : le site utilise un compte qui peut seulement lire, et la base n'est pas joignable depuis le réseau.

Complétez avant de configurer.

Élément	Valeur
Nom de la base	entreprise
Utilisateur dédié	webuser@localhost (connexion autorisée uniquement depuis le serveur lui-même)
Droits accordés	SELECT sur entreprise.* uniquement : le site ne fait que lire

☒ Installer MariaDB, lancer la sécurisation

Pourquoi ? *MariaDB stocke les données. Le script de sécurisation supprime les comptes anonymes, la base de test et l'accès root à distance : on part d'une base propre.*

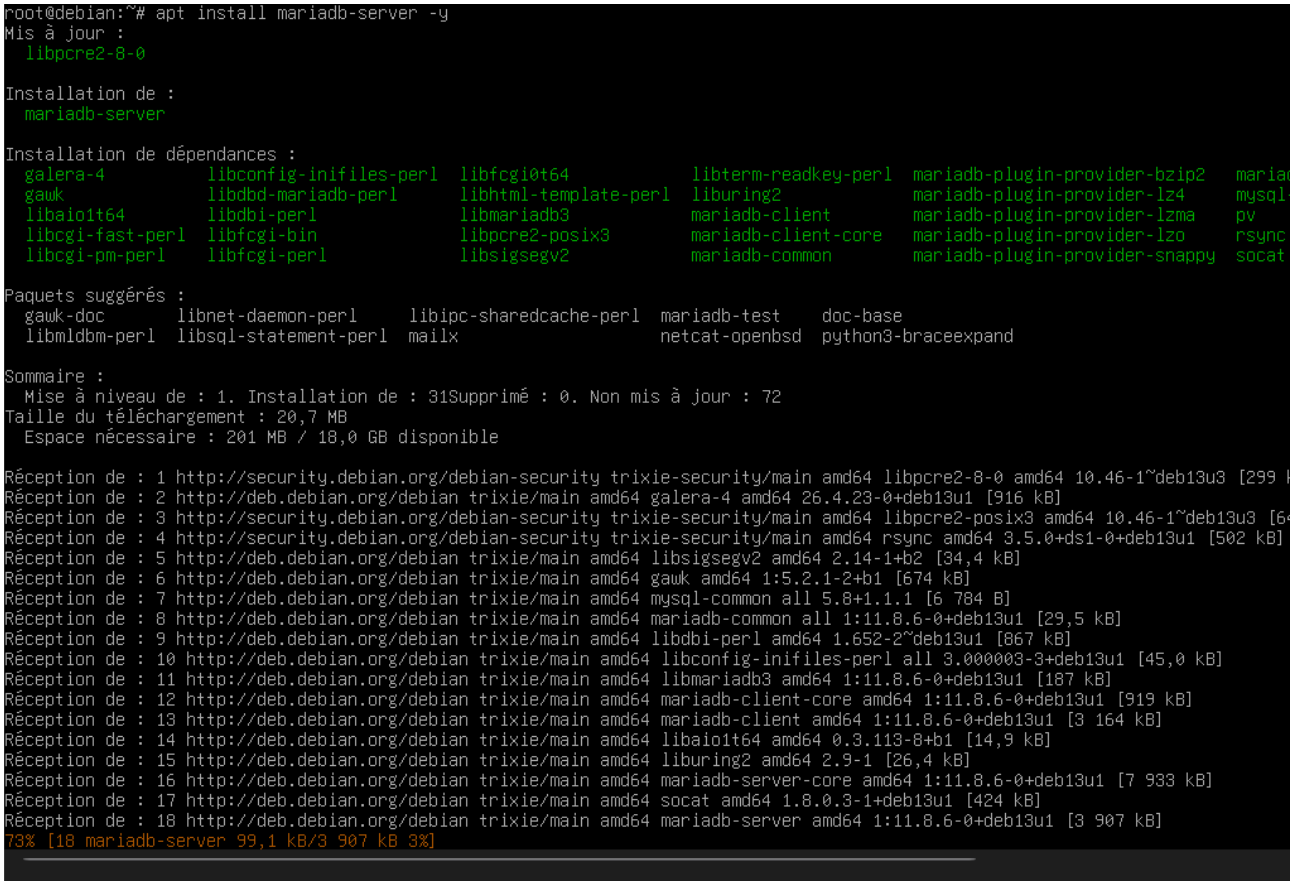


Figure 6 – Installation de MariaDB

La commande « apt install mariadb-server » installe le serveur de base de données MariaDB et ses dépendances. La sécurisation est ensuite faite avec « mariadb-secure-installation » (tableau ci-dessous).

Sécurisation :

Question	Réponse
Enter current password for root	Entrée (vide)
Switch to unix_socket authentication	n (root l'utilise déjà sur Debian)
Change the root password	n, ou Y si le formateur l'exige
Remove anonymous users	Y
Disallow root login remotely	Y
Remove test database	Y
Reload privilege tables now	Y

☒ Créer la base, l'utilisateur dédié et une table de 3 lignes

Pourquoi ? *Le site utilise un compte à part, webuser, qui peut seulement lire (SELECT). Même si le site était piraté, l'attaquant ne pourrait ni modifier ni supprimer les données.*

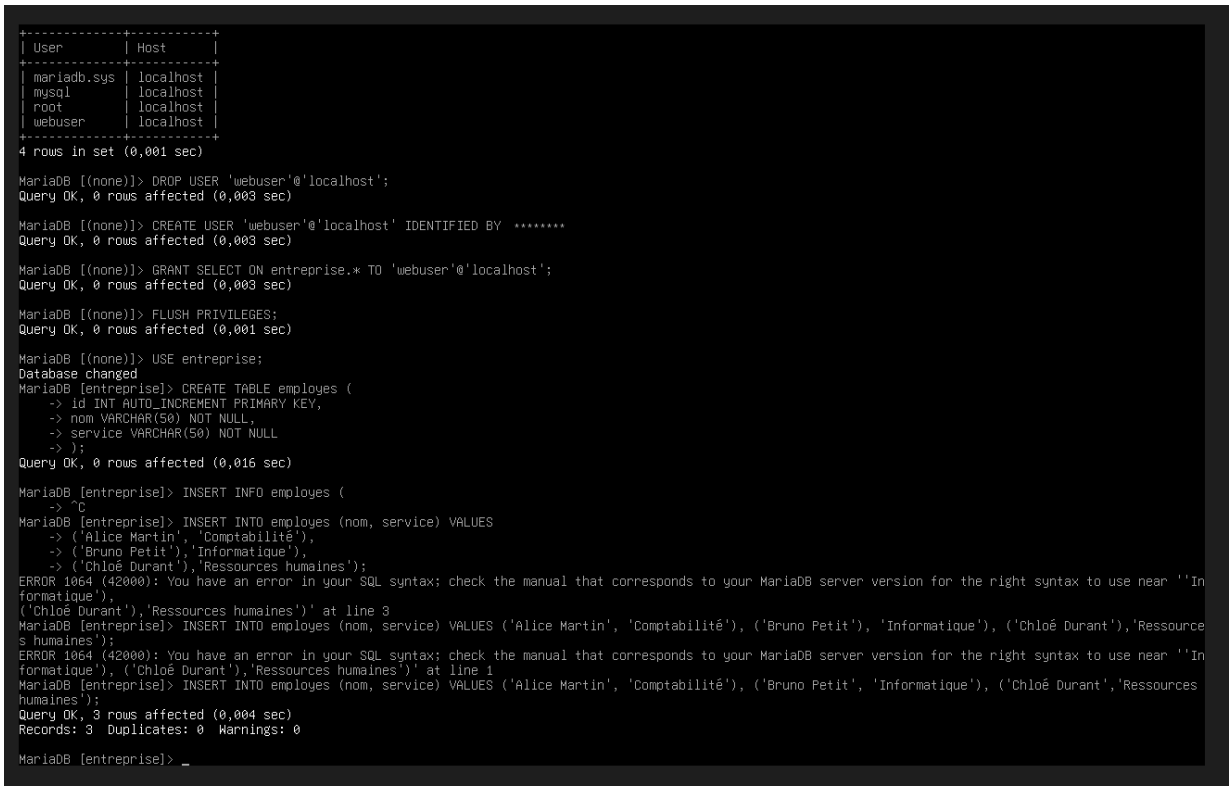


Figure 7 – Création de l'utilisateur, de la table et des données
On crée l'utilisateur dédié webuser, qui ne peut se connecter que depuis le serveur (localhost), et on lui donne uniquement le droit SELECT sur la base entreprise. On crée ensuite la table employes. Les deux premiers INSERT échouent (erreur 1064 : parenthèses mal placées) ; le troisième ajoute bien les 3 lignes.

```
MariaDB [entreprise]> SELECT * FROM employes;
+----+---------------+---------------------+
| id | nom           | service             |
+----+---------------+---------------------+
|  1 | Alice Martin  | Comptabilité        |
|  2 | Bruno Petit   | Informatique        |
|  3 | Chloé Durant  | Ressources humaines |
+----+---------------+---------------------+
3 rows in set (0,000 sec)

MariaDB [entreprise]> SHOW GRANTS FOR 'webuser'@'localhost';
+-----------------------------------------------------------------------------------------------------------------+
| Grants for webuser@localhost                                                                                    |
+-----------------------------------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `webuser`@`localhost` IDENTIFIED BY PASSWORD ********                                     |
| GRANT SELECT ON `entreprise`.* TO `webuser`@`localhost`                                                         |
+-----------------------------------------------------------------------------------------------------------------+
2 rows in set (0,000 sec)

MariaDB [entreprise]>
```

Figure 8 – Contrôle des données et des droits
« SELECT * FROM employes » affiche les 3 employés. « SHOW GRANTS » confirme que webuser n'a que le droit SELECT sur la base entreprise : c'est le principe du moindre privilège.

☒ Installer PHP, afficher la table dans une page (code fourni)

Pourquoi ? *PHP fait le lien entre Apache et MariaDB : il interroge la base, puis transforme le résultat en page HTML que le navigateur sait afficher.*

```
root@debian:~# apt install php libapache2-mod-php php-mysql -y
Installation de :
  libapache2-mod-php  php  php-mysql

Installation de dépendances :
  libapache2-mod-php8.4  php-common  php8.4  php8.4-cli  php8.4-common  php8.4-mysql  php8.4-opcache  php8.4-readline

Paquets suggérés :
  php-pear

Sommaire :
  Mise à niveau de : 0. Installation de : 11Supprimé : 0. Non mis à jour : 72
Taille du téléchargement : 5 126 kB
  Espace nécessaire : 24,4 MB / 17,6 GB disponible

Réception de : 1 http://security.debian.org/debian-security trixie-security/main amd64 php8.4-common amd64 8.4.26-1~deb13u1 [781 kB]
Réception de : 2 http://deb.debian.org/debian trixie/main amd64 php-common all 2:96 [13,3 kB]
Réception de : 3 http://deb.debian.org/debian trixie/main amd64 libapache2-mod-php all 2:8.4+96 [4 068 B]
Réception de : 4 http://deb.debian.org/debian trixie/main amd64 php all 2:8.4+96 [3 936 B]
Réception de : 5 http://deb.debian.org/debian trixie/main amd64 php-mysql all 2:8.4+96 [3 968 B]
11% [1 php8.4-common 233 kB/781 kB 30%]_
```

Figure 9 – Installation de PHP
On installe PHP, le module qui permet à Apache d'exécuter du PHP (libapache2-mod-php) et l'extension php-mysql, qui permet à PHP de dialoguer avec MariaDB.

```
  GNU nano 8.4                                                    /var/www/html/index.php
<?php
$db = new mysqli('localhost', 'webuser', 'webRP62026', 'entreprise');
$resultat = $db->query ('SELECT * FROM employes');
?>
<h1>Intranet - AIT TAHAR Menzo</h1>
<table border='1'>
<tr><th>ID</th><th>Nom</th>
<th>Services</th></tr>
<?php
while ($ligne = $resultat->fetch_assoc()) {
    echo "<tr><td>{$ligne['id']}</td><td>{$ligne['nom']}
</td><td>{$ligne['service']}<td></tr>";
}
?>
</table>
```

Figure 10 – Code de la page index.php
Le script se connecte à la base entreprise avec l'utilisateur webuser, lit la table employes, puis affiche chaque ligne dans un tableau HTML grâce à une boucle while. Sur cette capture, le mot de passe commence par une minuscule : cette erreur provoquait une erreur 500 et a été corrigée ensuite.

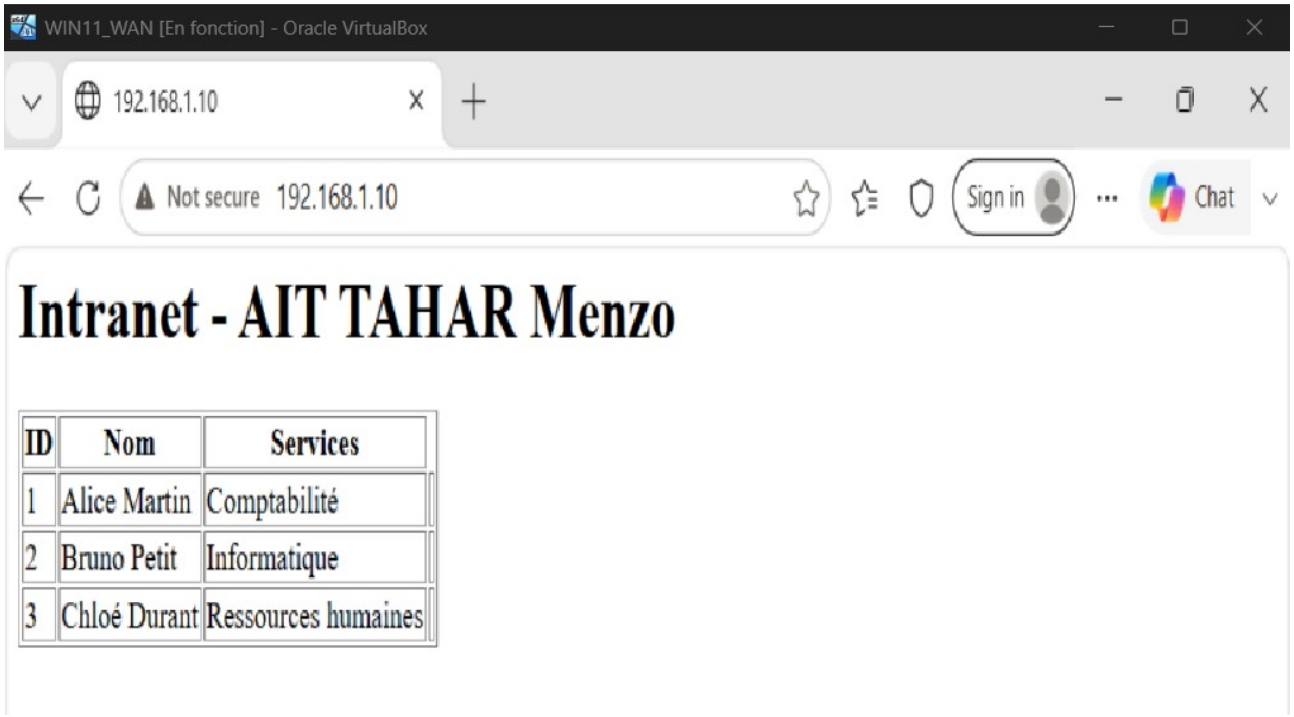


Figure 11 – Le site affiche la base de données
Depuis le client, http://192.168.1.10 affiche le tableau des 3 employés lu dans MariaDB. Le site fonctionne, mais il est encore en HTTP : le navigateur indique « Not secure ».

☒ Vérifier que la base n'écoute pas sur le réseau

Pourquoi ? *Seul PHP, sur le serveur lui-même, a besoin de la base. En la limitant à l'adresse locale 127.0.0.1, personne ne peut l'attaquer depuis le réseau.*

```
root@debian:~# ss -tlnp | grep 3306
LISTEN 0      80         127.0.0.1:3306      0.0.0.0:*    users:(("mariadbd",pid=3029,fd=28))

root@debian:~# grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf
bind-address            = 127.0.0.1
root@debian:~# _
```

Figure 12 – MariaDB n'écoute qu'en local
« ss -tlnp » montre que MariaDB écoute uniquement sur 127.0.0.1:3306, et le fichier 50-server.cnf contient « bind-address = 127.0.0.1 ». La base n'est donc pas accessible depuis le réseau : seul le serveur lui-même peut l'interroger.

6 Application C - HTTPS 1 h 10

BUT DE LA PHASE
Faire en sorte que rien ne circule en clair entre le navigateur et le serveur. Le site est chiffré en HTTPS avec un certificat signé par l'autorité de certification interne de l'entreprise, créée dans pfSense. Les postes qui font confiance à cette autorité n'affichent plus d'alerte.

☒ Activer HTTPS sur Apache avec le certificat par défaut

Pourquoi ? *HTTPS chiffre les échanges sur le port 443. Le certificat fourni par Debian permet de tester rapidement, mais il est auto-signé.*

Objectif : https://192.168.1.10 s'ouvre avec un cadenas sans alerte, grâce à un certificat signé par une CA créée dans pfSense, et http:// redirige vers https://.

```
root@debian:~# a2enmod ssl*
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Enabling module socache_shmcb.
Enabling module ssl.
See /usr/share/doc/apache2/README.Debian.gz on how to configure SSL and create self-signed certificates.
To activate the new configuration, you need to run:
  systemctl restart apache2
root@debian:~# a2enmod ssl
Considering dependency mime for ssl:
Module mime already enabled
Considering dependency socache_shmcb for ssl:
Module socache_shmcb already enabled
Module ssl already enabled
root@debian:~# a2ensite default-ssl
Enabling site default-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
root@debian:~# systemctl restart apache2
root@debian:~# ss -tlnp | grep ':443'
LISTEN 0      511                *:443            *:*    users:(("apache2",pid=10549,fd=6),("apache2",pid=10548,fd=6),("apache2
=10546,fd=6),("apache2",pid=10545,fd=6),("apache2",pid=10543,fd=6))
root@debian:~# _
```

Figure 13 – Activation de HTTPS
« a2enmod ssl » active le module SSL d'Apache et « a2ensite default-ssl » active le site HTTPS. Après le redémarrage, « ss » montre qu'Apache écoute aussi sur le port 443.

☒ Ouvrir le site : pourquoi le navigateur alerte-t-il ?

Pourquoi ? *Le navigateur alerte parce qu'aucune autorité qu'il connaît ne garantit ce certificat : il ne peut pas vérifier que le serveur est bien celui qu'il prétend être.*

Sur le client, ouvre https://192.168.1.10 (le port 443 doit être autorisé dans pfSense, d'où l'alias Ports_Web). Tu obtiens « Votre connexion n'est pas privée » avec deux erreurs possibles :

- NET::ERR_CERT_AUTHORITY_INVALID : le certificat est auto-signé, aucune CA connue du navigateur ne l'a signé.
- NET::ERR_CERT_COMMON_NAME_INVALID : il est émis pour le nom debian, pas pour l'IP tapée.

Capture cette alerte et clique sur « Avancé » pour voir le détail : c'est ta réponse à la question du sujet.

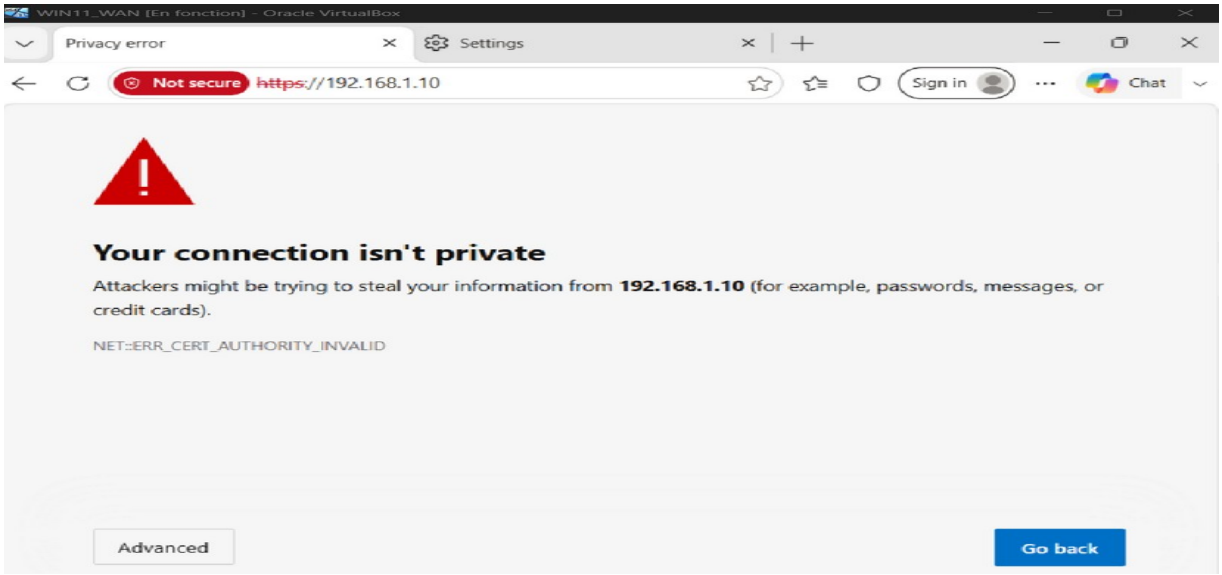


Figure 14 – Alerte du navigateur
En ouvrant https://192.168.1.10, Edge affiche « Your connection isn't private » avec l'erreur NET::ERR_CERT_AUTHORITY_INVALID. Le certificat par défaut d'Apache est auto-signé : aucune autorité de confiance ne garantit l'identité du serveur.

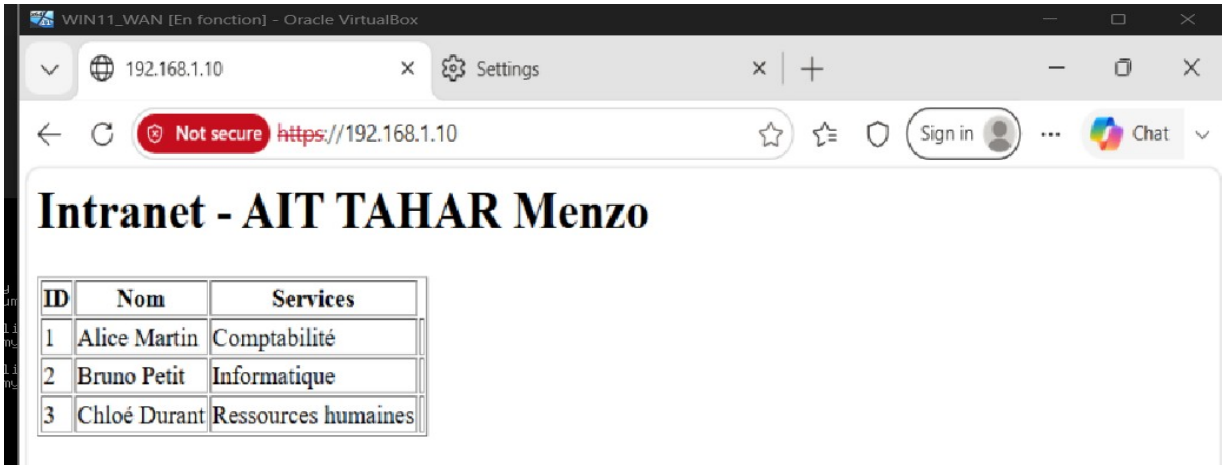


Figure 15 – Site chiffré mais non approuvé
Après avoir forcé l'accès (Advanced, puis Continue), la page s'affiche en HTTPS. La connexion est chiffrée, mais « Not secure » et « https » barré rappellent que le certificat n'est pas reconnu.

☒ Créer une CA dans pfSense, émettre un certificat pour l'IP du serveur

Pourquoi ? *La CA interne joue le rôle d'autorité de confiance de l'entreprise. Elle signe le certificat du serveur web, émis pour son adresse IP 192.168.1.10.*

Depuis le navigateur du client, dans l'interface de pfSense : **System > Certificates > Authorities > Add** (sur les anciennes versions : System > Cert. Manager > CAs).

Champ	Valeur
Descriptive name	CA-Entreprise
Method	Create an internal Certificate Authority
Key type / length	RSA, 2048
Digest Algorithm	sha256
Lifetime (days)	3650
Common Name	CA-Entreprise
Country, State, City, Organization	FR, ta région, ta ville, le nom de l'entreprise

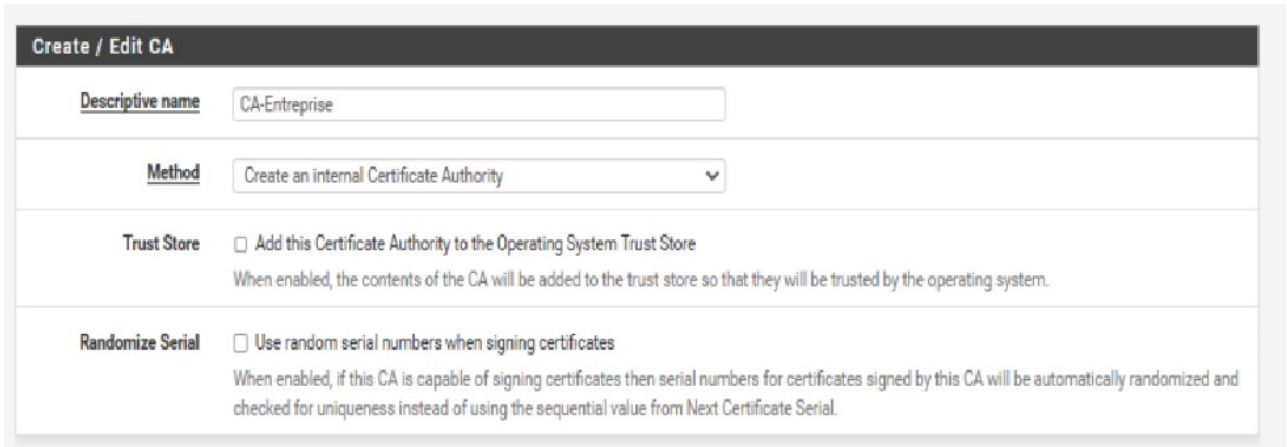


Figure 16 – Création de la CA dans pfSense (1/2)
Dans System > Cert. Manager > CAs, on crée une autorité de certification interne nommée CA-Entreprise. C'est elle qui signera le certificat du serveur web.

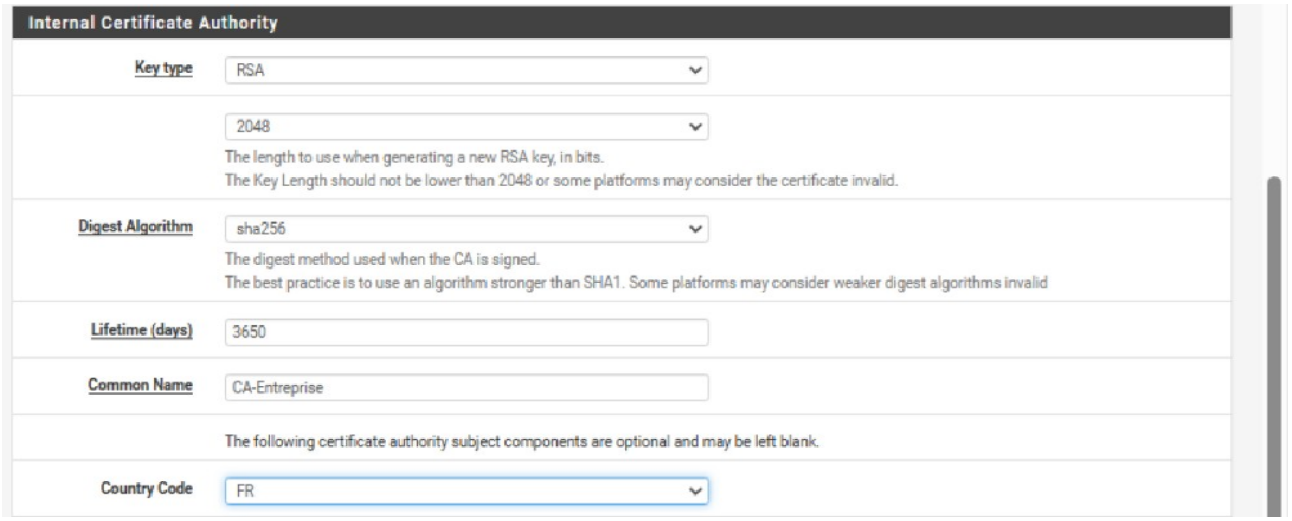


Figure 17 – Création de la CA dans pfSense (2/2)
La CA utilise une clé RSA de 2048 bits, l'algorithme sha256 et une durée de validité de 3650 jours (10 ans). Son nom commun (Common Name) est CA-Entreprise.

☐ Installer le certificat sur Apache, la CA sur le client

Pourquoi ? *Apache présente le certificat signé par la CA. Le client, qui fait désormais confiance à la CA, peut vérifier l'identité du serveur : l'alerte disparaît.*

Champ	Valeur
Method	Create an internal Certificate
Descriptive name	serveur-web
Certificate authority	CA-Entreprise
Key type / length, Digest	RSA 2048, sha256
Lifetime (days)	398
Common Name	192.168.1.10
Certificate Type	**Server Certificate**
Alternative Names	Type **IP address**, Value 192.168.1.10

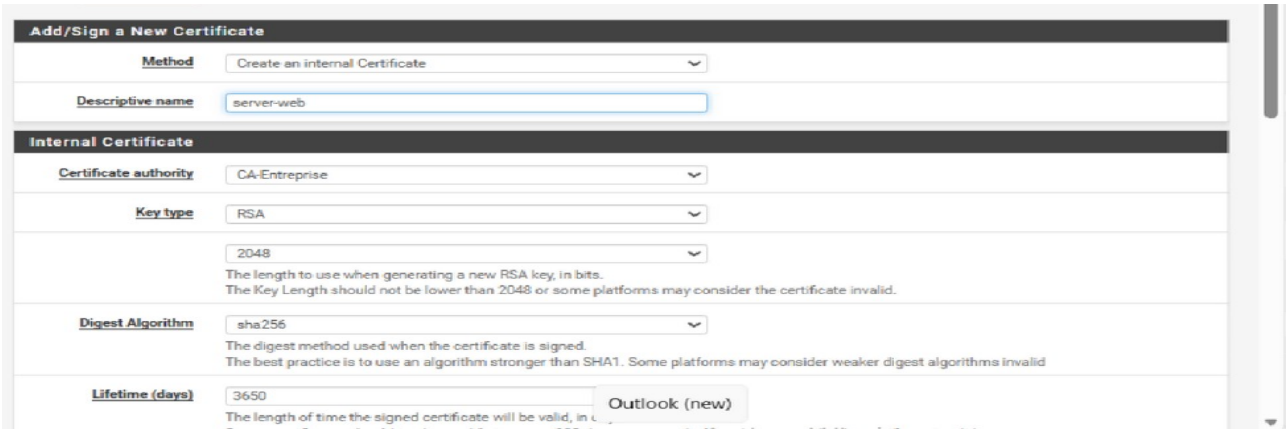


Figure 18 – Création du certificat du serveur
Dans l'onglet Certificates, on crée un certificat interne signé par CA-Entreprise (RSA 2048 bits, sha256). Cette capture montre le premier essai, nommé server-web.



Figure 19 – Certificat obtenu au premier essai
Le certificat est bien de type « Server Certificate » et signé par CA-Entreprise, mais il a été créé avec l'adresse d'exemple 192.168.20.10. Il a donc été supprimé puis recréé sous le nom serveur-web, avec le nom commun 192.168.1.10 et un nom alternatif (SAN) de type IP 192.168.1.10.

Le nom alternatif (SAN) de type IP est indispensable : Chrome et Edge ignorent le Common Name et ne vérifient que le SAN. Sans lui, l'alerte reste.

Save, puis exporte trois fichiers (ils arrivent dans les Téléchargements du client) :

- onglet Certificates, ligne serveur-web : **Export Certificate** (serveur-web.crt) et **Export Key** (serveur-web.key) ;
- onglet Authorities, ligne CA-Entreprise : **Export CA** (CA-Entreprise.crt).

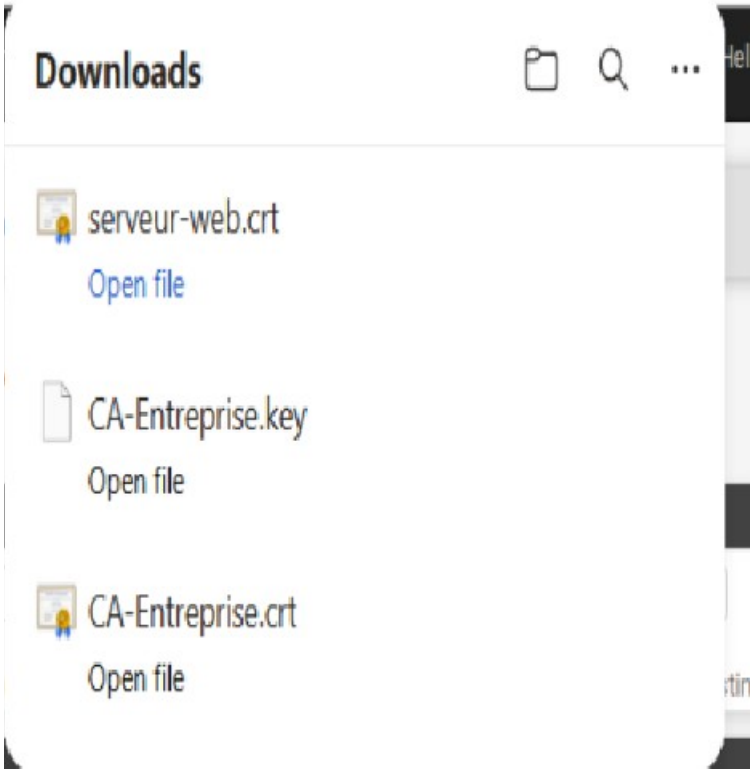


Figure 20 – Fichiers exportés depuis pfSense
Le certificat du serveur (serveur-web.crt) et celui de la CA (CA-Entreprise.crt) sont téléchargés sur le client. La clé privée de la CA (CA-Entreprise.key) a été exportée par erreur : elle ne doit jamais quitter pfSense et doit être supprimée du poste.

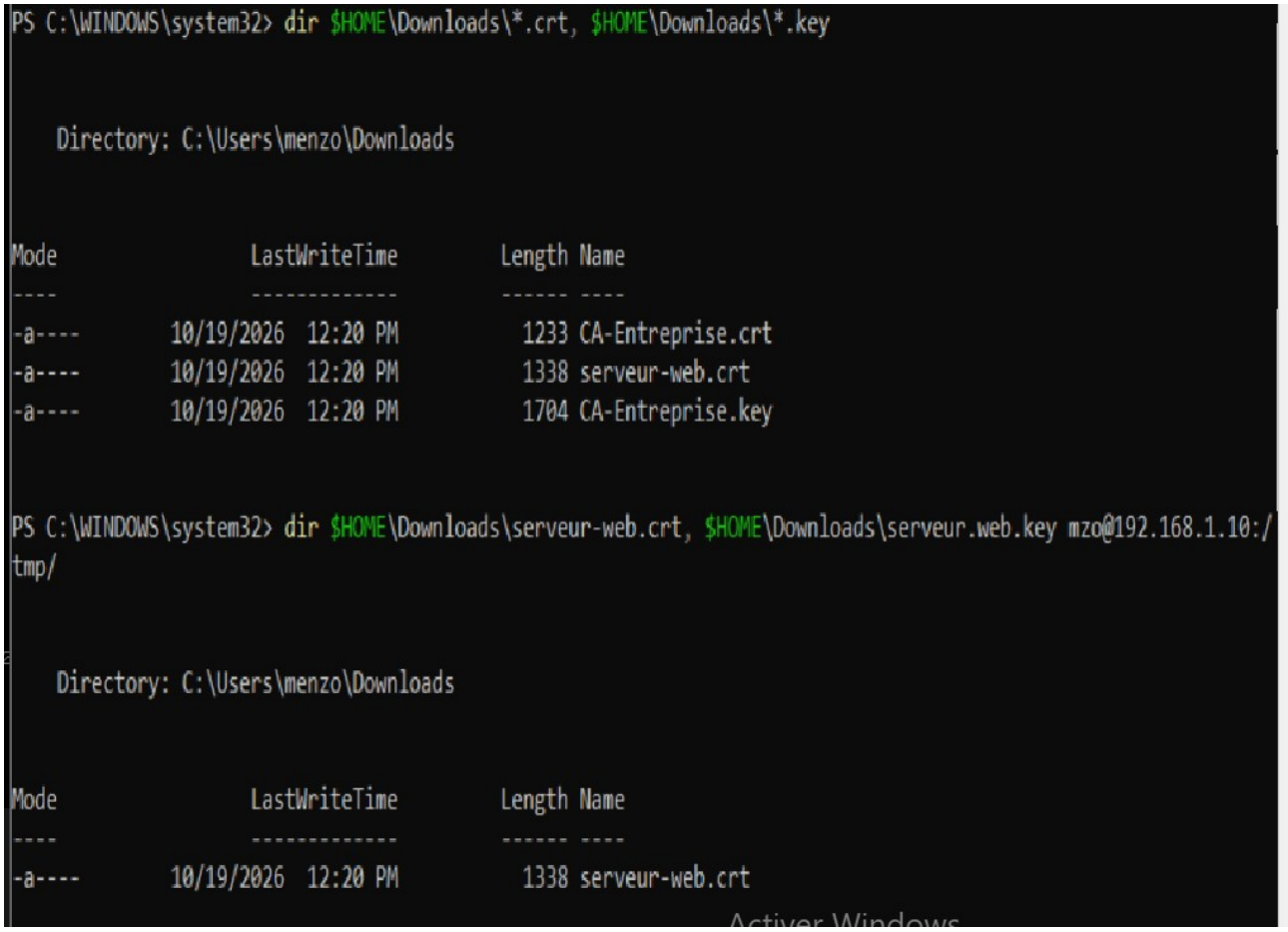


Figure 21 – Vérification des fichiers sur le client
La commande « dir » liste les certificats et les clés présents dans le dossier Téléchargements. À ce stade, la clé du serveur (serveur-web.key) n'avait pas encore été exportée.

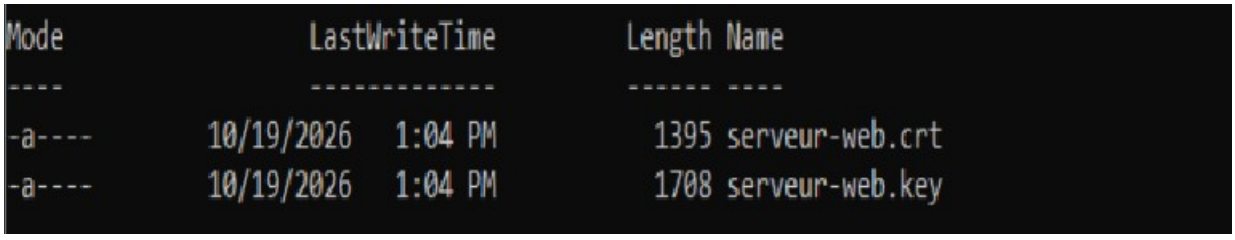


Figure 22 – Certificat et clé du serveur
Après un nouvel export depuis pfSense, le dossier contient les deux fichiers dont Apache a besoin : serveur-web.crt (le certificat) et serveur-web.key (sa clé privée).

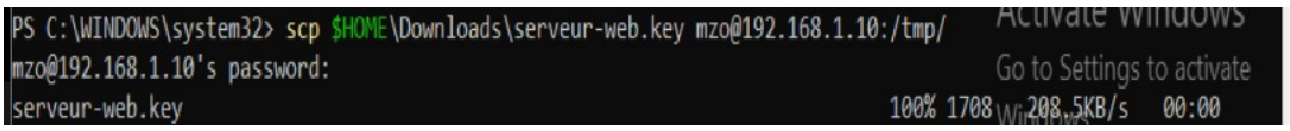


Figure 23 – Transfert vers le serveur
« scp » copie la clé serveur-web.key du client Windows vers le dossier /tmp du Debian, à travers une connexion SSH chiffrée. Le certificat a été envoyé de la même façon. Les deux fichiers sont ensuite rangés dans /etc/ssl/ et déclarés dans default-ssl.conf.

- ☐ Rediriger HTTP vers HTTPS, vérifier le cadenas

Pourquoi ? *La redirection oblige tous les visiteurs à passer en HTTPS, même s'ils tapent l'adresse sans « https:// ». Plus aucune page ne circule en clair.*

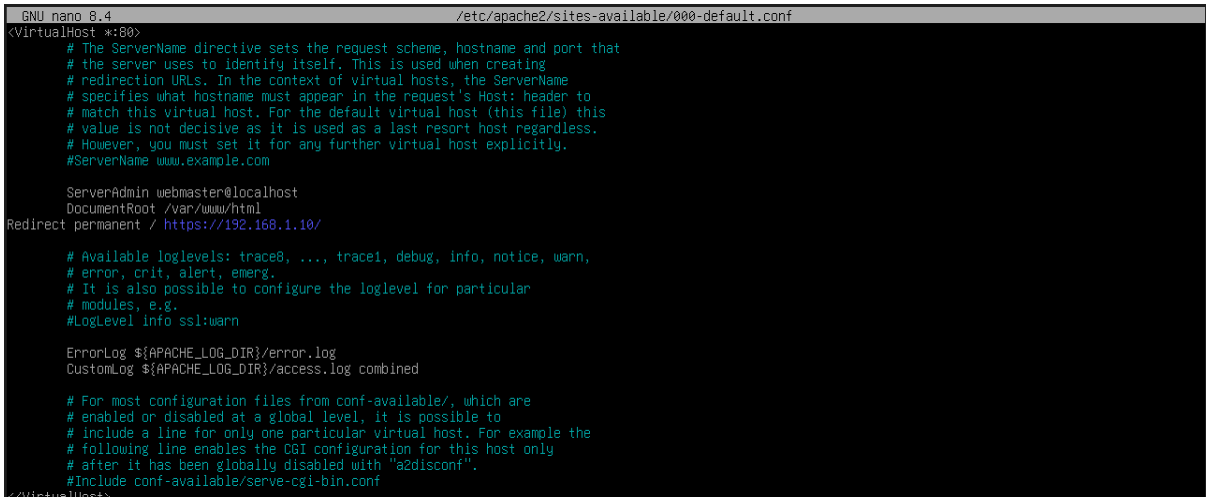


Figure 24 – Redirection de HTTP vers HTTPS
Dans 000-default.conf, le site du port 80, la ligne « Redirect permanent / https://192.168.1.10/ » envoie automatiquement tous les visiteurs HTTP vers la version HTTPS du site.

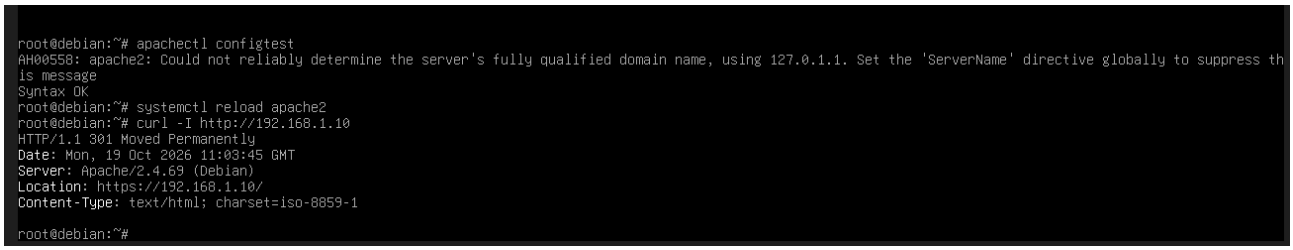


Figure 25 – Test de la redirection
« apachectl configtest » vérifie la configuration : « Syntax OK » (le message AH00558 n'est qu'un avertissement sur le nom du serveur). Après « systemctl reload apache2 », « curl -I http://192.168.1.10 » renvoie « 301 Moved Permanently » vers https://192.168.1.10/ : la redirection fonctionne.

7 Vérification — 20 min

BUT DE LA PHASE
Prouver que tout fonctionne ensemble : une ligne ajoutée dans la base apparaît sur le site dès qu'on recharge la page, en HTTPS et sans alerte. PHP relit la table à chaque affichage, donc le site est toujours à jour.

Le formateur ajoute une ligne dans la table. Elle doit apparaître sur le site, en HTTPS, sans alerte.

Capture à ajouter : la page https://192.168.1.10 sur le client, sans alerte, avec la ligne ajoutée par le formateur visible dans le tableau.

Rendu attendu

- ☐ La documentation NOM_Prenom_RP6.pdf, qui contient :
  - ☒ Le tableau de la base complété
  - ☐ Captures : la page qui affiche la table, le cadenas sans alerte
  - ☒ Un schéma simple : client / pfSense / Apache + MariaDB

☐ Les fichiers de configuration dans NOM_Prenom_RP6_config.zip : pfSense (config.xml), VirtualHost Apache HTTP et HTTPS (.conf), MariaDB (50-server.cnf)

☐ Le tableau de synthèse mis à jour avec cette RP

Le tableau de la base

Élément	Valeur
Nom de la base	entreprise
Utilisateur dédié	webuser@localhost : il ne peut se connecter que depuis le serveur lui-même
Droits accordés	SELECT sur entreprise.* uniquement (lecture seule)
Table	employes (id, nom, service), avec 3 lignes au départ

Ces choix appliquent le principe du moindre privilège : le site n'a que les droits dont il a besoin, c'est-à-dire lire la table.

Les captures demandées

Capture demandée	Où la trouver
La page qui affiche la table	Figure 11 (en HTTP) et Figure 15 (en HTTPS)
Le cadenas sans alerte	Capture à ajouter ci-dessous

Capture à ajouter : le site https://192.168.1.10 ouvert sur le client Windows, avec le cadenas sans alerte et le détail du certificat « Émis par : CA-Entreprise ».

Le schéma de la maquette

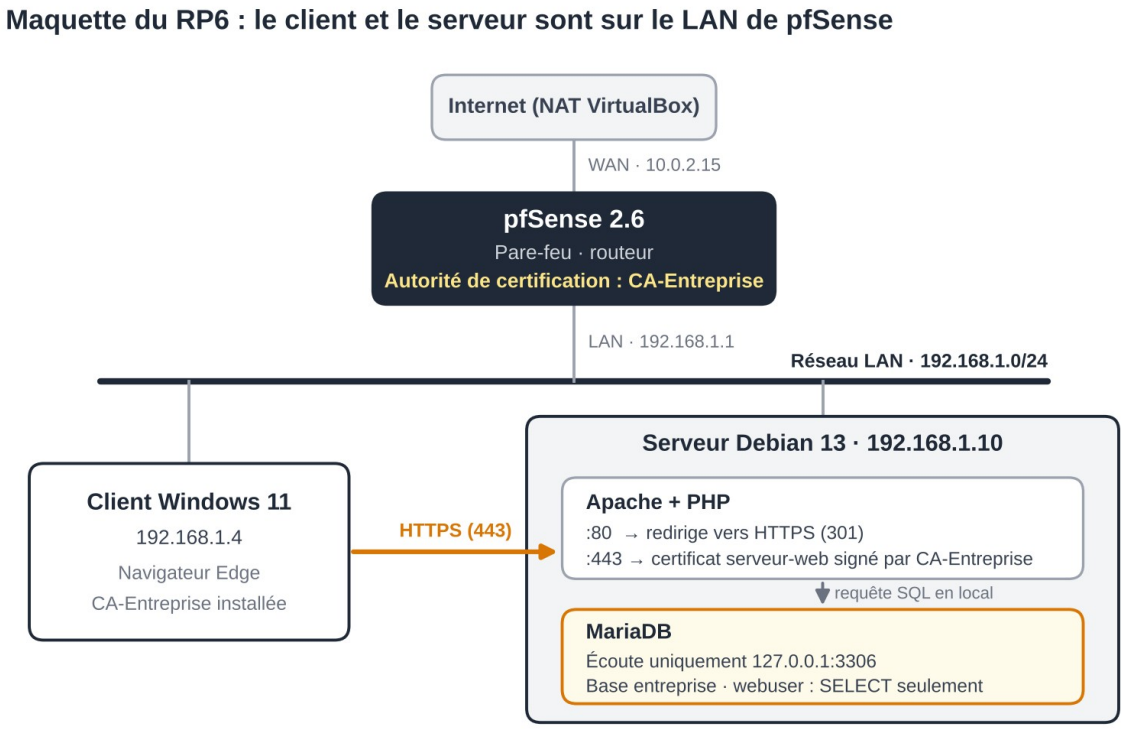


Figure 26 – Schéma de la maquette
Le client Windows 11 et le serveur Debian sont tous les deux sur le réseau LAN de pfSense (192.168.1.0/24). Le

client accède au site en HTTPS ; seul PHP, sur le serveur lui-même, interroge MariaDB. pfSense relie le LAN à Internet et héberge l'autorité de certification CA-Entreprise.

Les fichiers de configuration

Les fichiers suivants sont rassemblés dans l'archive AIT_TAHAR_Menzo_RP6_config.zip :

Fichier	Rôle	Emplacement
config.xml	Sauvegarde complète de pfSense : interfaces, règles, CA et certificats	pfSense : Diagnostics > Backup & Restore
000-default.conf	Site HTTP (port 80) : redirige vers HTTPS	/etc/apache2/sites-available/
default-ssl.conf	Site HTTPS (port 443) : utilise le certificat serveur-web	/etc/apache2/sites-available/
50-server.cnf	Configuration de MariaDB : bind-address = 127.0.0.1	/etc/mysql/mariadb.conf.d/

Remarque : config.xml contient la clé privée de la CA. Ce fichier ne doit être transmis qu'au formateur.

Le tableau de synthèse

Ligne à reporter dans le tableau de synthèse pour cette réalisation :

Rubrique	Contenu
Intitulé	RP6 – Serveur web Apache, base MariaDB et HTTPS avec une CA interne pfSense
Période	Octobre 2026
Modalité	TP en binôme sur une maquette virtualisée (VirtualBox)
Ressources	Debian 13, Apache 2.4, PHP 8.4, MariaDB 11.8, pfSense 2.6, Windows 11
Compétences mobilisées	Mettre à disposition des utilisateurs un service informatique ; gérer le patrimoine informatique ; sécuriser les services et les données (chiffrement, moindre privilège)

Conclusion

Ce TP m'a permis de mettre en place, de bout en bout, un site intranet sécurisé. Le serveur Debian héberge Apache et PHP, qui affichent les données d'une base MariaDB. Le site est chiffré en HTTPS grâce à un certificat signé par CA-Entreprise, l'autorité de certification interne créée dans pfSense, et toute connexion en HTTP est redirigée vers HTTPS.

Ce qui fonctionne

- Le site https://192.168.1.10 affiche le contenu de la table employes. Toute modification de la base apparaît dès qu'on recharge la page, car PHP relit la table à chaque affichage.
- Le compte webuser peut seulement lire la base, et MariaDB n'écoute que sur 127.0.0.1 : la base n'est pas joignable depuis le réseau.
- Les échanges entre le client et le serveur sont chiffrés, et le certificat du serveur est reconnu par le client, qui fait confiance à CA-Entreprise.

Difficultés rencontrées et solutions

Problème	Solution
La carte réseau du Debian reliée au LAN n'avait pas d'adresse IP : le client ne pouvait pas joindre le serveur.	Adresse fixe 192.168.1.10 configurée avec nmcli.
Copier-coller impossible dans la console VirtualBox.	Connexion en SSH depuis le client Windows.
Erreur 500 sur la page PHP (« Access denied » dans le journal d'Apache).	Le mot de passe écrit dans index.php ne respectait pas les majuscules : il a été aligné sur celui de la base.
Le premier certificat avait une mauvaise adresse IP, puis le type « User Certificate ».	Certificat recréé en « Server Certificate », avec 192.168.1.10 en nom commun et en nom alternatif (SAN).
La clé du serveur manquait lors du transfert (fichier non exporté, fautes de frappe dans scp).	Export de la clé depuis pfSense, vérification des noms avec dir, puis envoi avec scp.
Apache présentait toujours le certificat par défaut (CN=debian).	Chemins du certificat et de la clé corrigés dans default-ssl.conf, puis contrôle avec openssl.

Ce que j'ai appris

J'ai compris pourquoi on sépare le site et les données, pourquoi un service doit utiliser un compte aux droits limités, et comment fonctionne la confiance en HTTPS : un certificat n'est accepté que s'il est signé par une autorité que le poste connaît. J'ai aussi pris l'habitude de vérifier chaque étape avec des commandes de contrôle (curl, ss, openssl) et de lire le journal d'Apache pour trouver la cause d'une erreur.

Pistes d'amélioration

- Placer le serveur dans la zone SERVEURS de pfSense (192.168.2.0/24) et n'autoriser depuis le LAN que les ports 80 et 443 : le pare-feu contrôlerait alors réellement l'accès au site.
- Utiliser un nom, par exemple intranet.entreprise.local, avec un serveur DNS, plutôt qu'une adresse IP.
- Installer automatiquement la CA sur tous les postes, par exemple par stratégie de groupe dans un domaine.
- Sauvegarder régulièrement la base avec mariadb-dump.